

Federated Learning TP2

Adithya Prasad, Shivarpatna Venkatesh

Course: AI for Connected Industries

Module: Distributed and Federated Learning

Ulm University

Abstract—This report presents, the implementation and evaluation of TP2 from the Federated Learning course lab. The federated learning pipeline was developed using the Flower framework, as part of TP1 is leveraged to examine the implications of data heterogeneity and client drift as key challenges. Fedprox and scaffold methods were explored to mitigate the impact of heterogeneity on federated learning. The results show that SCAFFOLD provides the most consistent improvement under high heterogeneity, while benefit of FedProx is sensitive to the choice of μ .

Index Terms—Federated Learning, non-IID data, alpha Dirchlet, FedAvg, fedprox, scaffold

I. INTRODUCTION

Federated learning (FL) enables a global model to be trained across many clients, where each of them trains locally on their own partition and sends only model updates to a central server for global model aggregation. The key challenge in federated learning is client drift caused by non-IID data (heterogeneity). The concerns of heterogeneity are explored under experimental conditions to observe behaviors.

II. REPORT STRUCTURE

The end to end orchestration and experimentation of federated learning is hosted on GitHub for replicability. (Git Repo: dfl-tp-workspace). Report covers the results from different experiments on flower where various hyperparameter values are tested in combination. The plots from every tp experiments are recorded in a single pdf added as annexure (annexure-output-plots.pdf).

III. BACKGROUND

A. Fedprox

FedProx (Federated Proximal) is an extension of FedAvg designed to handle heterogeneous (non-IID) client data and variable local computation. It adds a proximal term to the local objective so that each client's update does not drift too far from the global model. This stabilizes training against client drift. Penalty added to objective function:

$$\min_w [F_k(w)] + \frac{\mu}{2} \|w - w^t\|^2 \quad (1)$$

where μ is the strength of penalty.

B. SCAFFOLD

SCAFFOLD (Stochastic Controlled Averaging) corrects for client drift by using control variates. Each client and the server maintain correction vectors that estimate the difference between local and global gradients. These corrections ensure that updates move in the right direction even when data is highly non-IID. Weights update with control variate:

$$w_{t+1}^k = w_t^k - \eta(g_k(w_t^k) - c_k + c) \quad (2)$$

where c_k = local control variates and c = server control variates

IV. EXPERIMENTS

TABLE I: Default Setup: Fixed and varied parameters for all experiments.

Parameter	Value
Dataset	FashionMNIST
Model	CNN (2 conv layers + 2 FC layers)
Learning rate	0.01
Clients n	50
Fraction train	1
Local epochs E	3
Batch size	64
Communication rounds	10
Varied	
alpha	$\alpha \in \{0.01, 5, 100\}$
Baseline	FedAvg+SGD
Mitigation (proximal parameter)	FedProx ($\mu=0.01$)
Mitigation (Control Variates)	Scaffold

A. Effect of data heterogeneity (α) on FedAvg Convergence: Baseline with no mitigation

1) *Expected Behavior*: FedAvg convergence should improve monotonically with α . Near-IID data ($\alpha = 100$) should yield the fastest convergence and accuracy because client gradients point in similar directions, unlike non-IID data ($\alpha = 0.01$).

2) *Observation and Deviations*:

- $\alpha = 1$ achieves the highest final central accuracy despite having moderate heterogeneity. (refer Table II)
- $\alpha = 0.01$ The model overfits on the local data as training occurs only on few dominant classes. (refer Figure 2)
- **Aggregation noise at large n .** With $n = 50$ clients and $\alpha = 100$, produces high-variance due to aggregation of noise from each client, leading to slower convergence than the moderate heterogeneity case ($\alpha = 1$), where clients carry more complementary class information. (refer Figure 1)

TABLE II: Final round metrics for each experiment with aggregation method and α setting.

Config	Train Loss	Train Acc	Test Loss	Test Acc
baseline ($\alpha=0.01$)	0.008	99.81%	1.263	58.16%
baseline ($\alpha=1$)	0.316	88.54%	0.626	76.83%
baseline ($\alpha=100$)	0.412	84.64%	1.574	71.6%
fedprox ($\alpha=0.01$)	0.007	99.82%	1.512	47.05%
fedprox ($\alpha=1$)	0.323	88.32%	0.661	75.02%
fedprox ($\alpha=100$)	0.434	83.83%	1.549	71.51%
scaffold ($\alpha=0.01$)	0.008	99.82%	1.099	61.32%
scaffold ($\alpha=1$)	0.304	89.02%	0.607	77.46%
scaffold ($\alpha=100$)	0.411	84.43%	1.611	71.59%

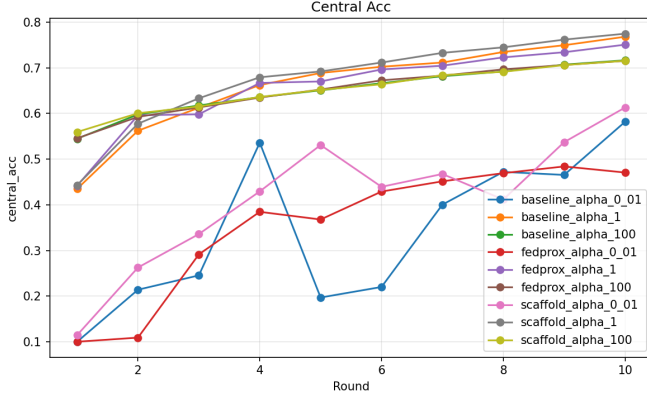


Fig. 1: Central Test Accuracy for $\alpha \in \{0.01, 1, 100\}$ with 50 clients

B. Experiment: Implementing FedProx and Scaffold Strategies

1) *Expected Behavior:* FedProx expected to outperform FedAvg under high heterogeneity using the proximal penalty $(\mu/2)\|w - w_{\text{global}}\|^2$ in the objective, which steers local weights towards the global model. SCAFFOLD is expected to provide the strongest improvement at low α by directly correcting the gradient bias of each client using control variates c_k and c , without slowing local training. Near IID all 3 methods are expected to have similar performance.

2) Observation and Deviations:

- FedProx under performs under $\alpha = 0.01$ conditions (refer Figure 1). At $\alpha = 0.01$ the global model is a poor initializer, hence the proximal term anchors local training to it, preventing clients from making the productive class-specific updates that FedAvg allows freely. Tuning of μ could possibly rectify the difference with fedavg.
- **SCAFFOLD provides marginal benefit at $\alpha = 0.01$.** The control variates generalizes the model better and keeps all client gradients closer to global optimum. This leads to slightly quicker convergence and compared to fedprox has marginally better performance in terms of accuracy. (refer Table III)
- **All three algorithms converge to similar accuracy at $\alpha = 100$.** FedAvg, FedProx, and SCAFFOLD all reach similar performance which is consistent with theory

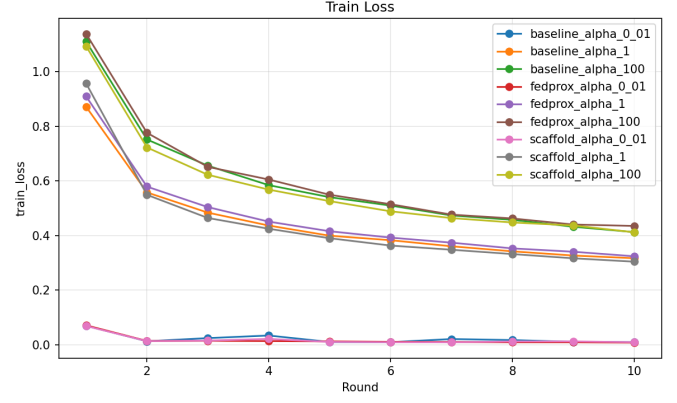


Fig. 2: Train Loss for $\alpha \in \{0.01, 1, 100\}$ with 50 clients

TABLE III: FedAvg vs FedProx vs SCAFFOLD in non-IID condition

Algorithm	Convergence Speed to 40%	Stability
FedAvg (baseline)	7 rounds	unstable
FedProx	6 rounds	stable
SCAFFOLD	4 rounds	relatively stable

for near-IID data, as client gradients are more oriented towards global optimum. There is minimum correction offered by fedprox and scaffold in this scenario.

V. CONCLUSION

Client drift occurs because FedAvg's local optimization has no constraint on how far clients can move from the global model especially under high heterogeneity which compound this divergence. Each step moves clients further toward their local optimum and away from the global objective.

FedProx addresses drift passively by adding a proximal term that penalizes large deviations from the global model during local training. It behaves identically to FedAvg when heterogeneity is low (the penalty term is near zero) and provides increasing benefit as alpha decreases. The μ hyperparameter requires tuning as too small and the effect is negligible, too large and clients cannot learn their local data effectively.

SCAFFOLD addresses drift actively by maintaining control variates, estimates of each client's gradient bias relative to the global gradient. By correcting the gradient direction during local training, SCAFFOLD enables more stable convergence under heterogeneity. Under high non-IID (alpha=0.01), SCAFFOLD outperforms both FedAvg and FedProx in convergence speed and final accuracy. The cost is additional communication per round of control variates.

When data heterogeneity is low, all three algorithms perform similarly, and FedAvg is the pragmatic choice. As heterogeneity increases, FedProx and SCAFFOLD provides incremental improvement, but the appropriate choice depends on the communication budget and deployment constraints.